

K-Food Safety Watch

A Step-by-Step Guide

K-식품안전 위치: 비개발자를 위한 단계별 안내서

How a food-safety scientist and Claude built a working app in one morning, what each piece does, how to run it yourself, and how to keep going without becoming a programmer.

K-식품안전 위치

English

식약처 공공데이터(회사·판매증지, 검사부적합)를 한 화면에서 검색하고 지도로 봅니다. 영문 번역과 위해 분류, 심각도는 Claude가 생성한 참고 정보입니다.

10

전체 사례

5

회사·판매증지

5

검사부적합

5

심각도 높음

4

사·도 수

제품명·업소명·사유 검색

전체 출처

전체 위해 유형

전체 심각도

전체 지역

초기화

업소 소재지 지도 (사·군·구 단위)

+

-

Leaflet

Boundaries: KOSTAT via southkorea-maps, Tiles © Esri — Esri, HERE, Garmin, OpenStreetMap contributors

● 높음

● 보통

● 낮음

● 미평가

좌표는 주소의 사·군·구 중심점이며, 겹침을 피하려고 약간 흘트렸습니다. 지도에 없는 사례는 주소가 비어 있거나 좌표를 찾지 못한 경우입니다.

위해 유형별 건수

중금속		3
이물		3
사용불가 원료		2
잔류농약·동물용의약품		2

막대를 클릭하면 해당 유형으로 필터링됩니다.

사례 목록 (10)

● 높음

● 사용불가 원료

회사·판매증지

2026-10-08

경기도 고양시

알로에 추출물 분말

주식회사 성원에프아이

식품원료로 허용되지 않는 부위(껍질, 주스, 라텍스) 포함

시 요약

Written in Hongdae, Seoul, 9 October 2026

Bob Currier (OCEANCODA) with Claude Code

Repository: github.com/robertdcurrier/korean_food_safety (this guide is [docs/K-Food-Safety-Watch-Guide.pdf](#) there)

Contents

1. Who this is for, and the one lesson that matters most
2. What we built
3. The two Claudes: the builder and the worker
4. How the app works, in plain language
5. How it was built: the method
6. Setting up your computer
7. Getting your two keys
8. Working with Claude Code on this project
9. Exercises, in order of difficulty
10. Beyond food safety: the same recipe in any field
11. When something goes wrong
12. Glossary

SIXTY-SECOND VERSION

A Python program fetches Korean food recalls and failed inspections from the government's open API. It sends them, twenty at a time, to Claude, which translates them into English, classifies the hazard, judges the severity, and writes a one-sentence summary in both languages. The program then writes one HTML file you open in a browser: a searchable, filterable, bilingual map and list. Claude Code wrote every line of that program from plain-English conversation. You can run it with no keys in five minutes, and each key you add makes the result better.

1. Who this is for, and the one lesson that matters most

You are a scientist. You know what a recall grade means, which test items matter, and when a result is alarming versus merely out of spec. You do not need to become a software developer to build tools that use that knowledge. This guide shows you exactly how one such tool was built, and how to run and extend it.

THE KEY LESSON: OWN THE QUESTION, NOT THE KEYBOARD

Everything technical in this repository, the terminal, Git, Python, API keys, debugging a map that would not load, is routine work for someone who grew up with it. For a Gen MZ partner with even modest coding exposure, setting this up is an hour, not a weekend.

What that partner cannot supply is you: knowing that aloe latex contains laxative anthraquinones, that a grade-1 recall is a different animal from a grade-3, that "금속성이물 39.84 mg/kg against a 10 mg/kg limit" is serious, and which of the MFDS's hundreds of data services actually matter. That is the scarce input. The code is not.

So the strong recommendation is: find a partner who is comfortable in a terminal, hand them Sections 6 and 7 of this guide, and spend your own time on Sections 3, 5, 8 and 9. Decide what to build, judge whether the AI's output is right, and write the evaluation. Let them type.

Everything below is still written so you can do it alone if you want to. Treat the setup chapter as a test: if it takes you more than one evening, that is the signal to pair up, not to quit.

What you will be able to do after reading this

- Explain what the app does and what each of the two Claudes contributes.
- Run the app on your own computer and open the result.
- Obtain and install the two keys that unlock the full data and the AI features.
- Open Claude Code in the project and ask for changes in a way that gets good results.
- Judge whether the AI's classifications are right, which only you can do.

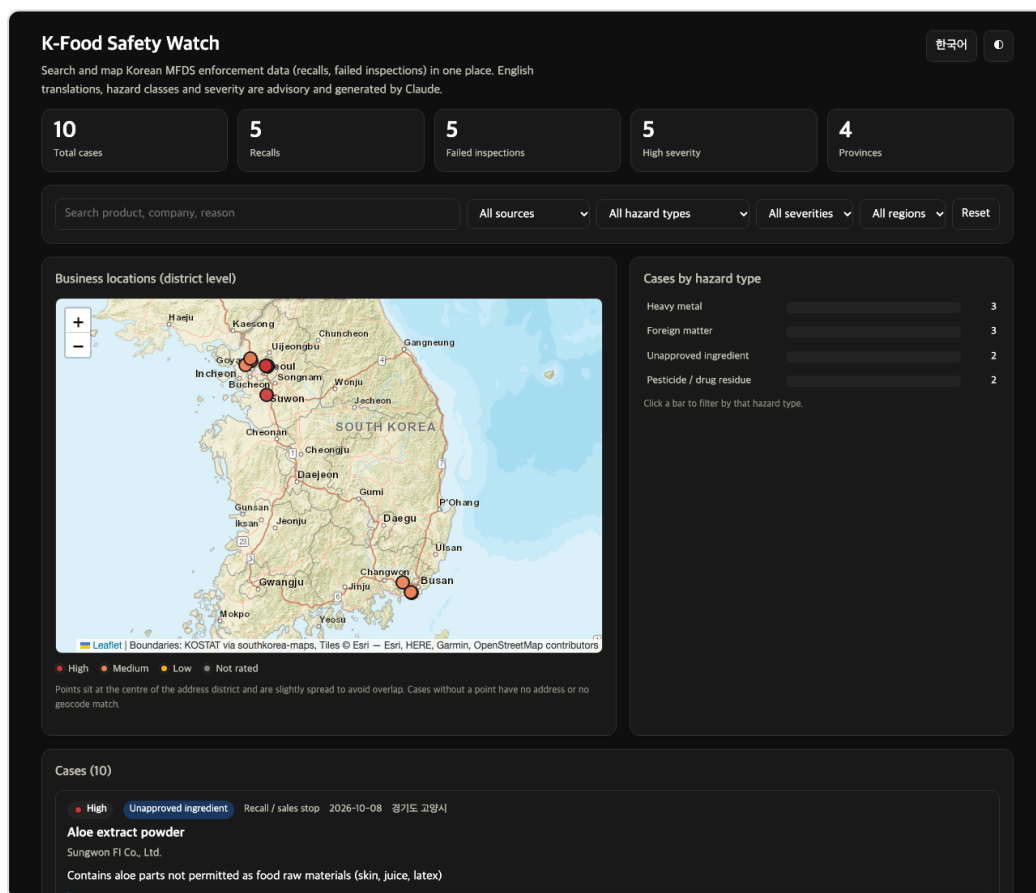
2. What we built

A single web page, saved as a file on your computer, that opens in any browser without a server. It shows Korean food-safety enforcement records from two sources published by the Ministry of Food and Drug Safety (식품의약품안전처, MFDS) through its open-data portal 식품안전나라:

Service	Korean name	What it contains	Records on 9 Oct 2026
I0490	회수·판매중지 정보	Products recalled or pulled from sale, with the reason and recall grade	383
I2620	검사부적합(국내)	Domestic lab inspections that failed a standard: the test item, the measured result, the limit	544

For every record, Claude adds an English translation, one of ten hazard categories, a severity of high, medium or low with a stated reason, and a one-sentence plain-language summary in Korean and in English.

The page has a Korean/English toggle, a light/dark toggle, free-text search, filters for source, hazard type, severity and province, a map with one marker per business coloured by severity, a bar chart of hazard types you can click to filter, and a full case list with every original MFDS field.



The same page in English, dark mode. Nothing is translated by the browser; every label exists in both languages inside the file.

What it deliberately is not

- Not a copy of the MFDS site. The MFDS lists each service separately, in Korean only, with no map, no hazard taxonomy, and no severity view.
- Not an official source. AI output is advisory and the page footer says so. The original record is always one click away.
- Not street-level. Markers sit at the centre of the business's district (시·군·구), slightly scattered so they do not overlap.

3. The two Claudes: the builder and the worker

This project uses Claude in two completely different ways, and keeping them apart in your head is the single most useful piece of understanding in this guide.

Claude Code: the builder

An application that runs on **your computer**, in the terminal. You talk to it in plain language. It reads and writes files, runs commands, searches the web, and asks you questions when it needs a decision.

It wrote every file in this repository: the plan, the Python code, the web page, the tests, the documentation, and this guide. It also created the GitHub repository and pushed to it.

Who talks to it: you.

What it produces: files and commands.

How you pay: a Claude Pro or Max subscription, or an API key billed by usage.

When it runs: only while you are working with it.

The Claude API: the worker inside the app

A service on **Anthropic's servers** that the finished app calls over the internet. It never sees your terminal, never writes a file, and has no memory between calls.

One file in this project,

`src/kfs/enrich.py`, sends it batches of twenty records with a set of instructions, and receives back a strictly structured answer for each record: translation, hazard, severity, reasons, summaries.

Who talks to it: the Python program.

What it produces: data, as JSON.

How you pay: an API key, billed per token.

When it runs: each time you run `python -m kfs enrich` on new records.

What the worker is actually asked to do

The instructions it receives (the "system prompt") are plain English and live in `src/kfs/enrich.py`. The essence:

You are a bilingual (Korean/English) food-safety analyst helping make public MFDS enforcement records readable. For each input record produce one output item with the same id. Translate product, company and reason into natural English. Choose one hazard category from: microbiological, chemical residue, heavy metal, foreign matter, allergen labelling, unapproved ingredient, additive violation, quality/spec, labelling (other), other. Judge severity as a consumer-health question: high for pathogens, toxins, undeclared allergens, sharp or hard foreign matter, banned substances, large heavy-metal exceedances, recall grade 1 등급; medium for residue or additive limits exceeded, recall grade 2 등급; low for quality or labelling faults, grade 3 등급. Write one sentence a shopper would understand, in Korean and in English. Do not invent facts. If the reason is ambiguous, say so and pick the more cautious severity.

And here is one real answer it gave, for the first record in the committed data:

```

product_en:      Aloe extract powder
company_en:      Sungwon FI Co., Ltd.
reason_en:       Contains aloe parts not permitted as food raw
                  materials (skin, juice, latex)
hazard:          unapproved_ingredient
severity:         high
severity_why_en: Grade 1 recall; aloe latex contains laxative
                  anthraquinones and is not allowed in food.
summary_ko:      주식회사 성원에프아이의 '알로에 추출물 분말'에 식품원료로
                  허용되지 않는 알로에 껍질·주스·라텍스 부위가 포함되어 회수됩니다.
summary_en:      Sungwon FI's aloe extract powder is being recalled
                  because it contains aloe skin, juice and latex,
                  which are not permitted as food ingredients.

```

YOUR JOB IN THIS PICTURE

The hazard categories and the severity rubric in that prompt were written by Claude Code in about a minute, from general knowledge. They are reasonable. They are not *yours*. Reading them critically, tightening them, and checking twenty of the model's answers against your own judgement is exactly the work only a subject-matter expert can do, and it is Exercise 5 in Section 9.

Cost, in real numbers

Run	Tokens in / out	Approximate cost (Claude Opus 5.5, Oct 2026 list price)
Ten sample records (what is committed)	2,831 / 4,217	about US\$0.10
Full dataset, about 930 records	roughly 90× that	about US\$9
Re-running with nothing new	0 / 0	nothing: results are cached by record id

4. How the app works, in plain language

The program is a pipeline of four steps. Each one reads a file and writes a file. You can run them together or one at a time, and each skips work it has already done.

1 · fetch

Asks the MFDS API for every row of both services and saves them. Converts the Korean field codes into one clean record format.

2 · enrich

Sends new records to Claude, twenty per call, and saves the structured answers in a cache keyed by record id.

3 · geocode

Splits each address into province and district, looks the district up once on OpenStreetMap's free geocoder, and caches the coordinates.

4 · build

Pours the records, the AI fields, the coordinates and Korea's province outlines into one HTML template and saves `output/index.html`.

The design rule behind it

AI FOR JUDGEMENT, CODE FOR RULES

Only step 2 uses AI. Parsing a date like (소비기한) 2031.8.13, splitting 경기도 고양시 일산동구 문봉길 6-37 into province and district, and drawing the page are all mechanical. They are written as ordinary code with tests, which makes them free, instant, and verifiable. Claude is spent only where a rule cannot do the job: understanding what a Korean recall reason means and how worried a consumer should be.

Ask this of every step in any AI project: *could a regular expression do this?* If yes, do not spend tokens on it.

Why it works with no keys

The MFDS portal accepts the literal word `sample` as a key and returns five fixed rows per service. If no Anthropic key is present, the enrich step prints a message and is skipped. So `python -m kfs all` produces a real, working page within minutes of cloning, just a small one with no AI fields. That was a deliberate choice: the first run should never fail. Then each key you add visibly improves the result, which is a far better way to learn what each key is for.

Where things live

Path	What it is
<code>PLAN.md</code>	The plan, written before the code. Goal, scope, decisions, open questions.
<code>CLAUDE.md</code>	Standards and working method that Claude Code reads at the start of every session.
<code>docs/METHODOLOGY.md</code>	The working method in ten steps, with the prompts that worked.
<code>docs/PROMPT_LOG.md</code>	The actual prompts that produced the repository.
<code>docs/MFDS_API.md</code>	The API as observed: URL shape, result codes, Korean field dictionary.
<code>src/kfs/</code>	The program: <code>mfds.py</code> , <code>enrich.py</code> , <code>geocode.py</code> , <code>report.py</code> , <code>cli.py</code> .
<code>templates/report.html</code>	The page. Every label in both languages in one table near the top.
<code>data/</code>	Raw downloads, clean records, the AI cache, the geocode cache, province outlines.
<code>output/index.html</code>	The result. Double-click it.
<code>.env</code>	Your keys. Git ignores this file; it never leaves your computer.

5. How it was built: the method

The entire repository came from two human messages and about ninety minutes of Claude Code working, plus a short exchange when the map misbehaved. Here is what the human did, because that is the part to copy.

1 Started with a briefing, not a task.

The first message said where we were, who the colleague is, what he knows, what he wants to learn, and what the deliverable looks like. It ended with "read this, and then report back". No code was requested. Claude Code came back with its understanding, a proposed layout, and three open questions. Correcting a misunderstanding at that point costs one sentence.

2 Decided only what a human must decide, and said so.

The second message chose the data source and the bilingual requirement, and explicitly delegated the rest: "I'm not sure of the format of the MFDS, so I'll rely on your judgement." Clear delegation produces better work than vague delegation.

3 Let the model probe reality before designing.

Claude Code searched for the API documentation, then fetched real rows with `curl`. That is how we learned in minutes that the sample key returns a fixed five rows, that field names are Korean abbreviations, and that inspection records often carry only a district-level address. Each fact changed the design.

4 Wrote the plan into the repository.

`PLAN.md` exists before the code and travels with it.

5 Put the standards where the model reads them every time.

`CLAUDE.md` holds the code rules and the working method. A standard that lives only in your head is one the model cannot follow.

6 Built in small steps that each produce a file you can open.

Fetch, enrich, geocode, build. When something is wrong you can see which file is wrong.

7 Looked at the result with real eyes.

The page was rendered and inspected before the first commit. Later, when the human opened it in a real browser, the map tiles were blocked by OpenStreetMap's usage policy. The fix was not to find another tile host to depend on but to embed Korea's province outlines in the page so a map always draws, and to treat online tiles as a bonus. Removing a dependency beats swapping it.

8 Committed the results, not just the code.

The data and the finished page are in the repository, so anyone can see the outcome without running anything.

9

Kept the prompts.

`docs/PROMPT_LOG.md` . Six months from now it will explain the repository better than the git history does.

THE ACTUAL SECOND PROMPT, VERBATIM

"Let's go with the Korean MFDS open data portal, as he mentioned that as a source, and it will show him how to connect and use an API. What Claude does: I'm not sure of the format of the MFDS, so I'll rely on your judgement to create a simple, yet useful demonstration that is not replicated on the MFDS site. Perhaps a simplified search tool, or something along that line? Maybe even with an integrated map of events/problems/etc? And yes, Hangul and English easily switchable, please."

Notice: no technical vocabulary at all. Context, one decision, one explicit delegation, two soft suggestions, one hard requirement.

6. Setting up your computer

Allow one evening. Every step has a check so you know it worked before moving on. Commands are shown for macOS (Terminal) and Windows (PowerShell); use the column for your machine. If a step fails, Section 11 has the common causes, and this is also the chapter to hand to a partner.

1 Open a terminal.

macOS: press `⌘ Space`, type *Terminal*, press Enter. Windows: press `⌘`, type *PowerShell*, press Enter. You will type commands into this window and press Enter after each one.

2 Install Git.

macOS: type `git --version`; if a dialog offers to install developer tools, accept it, wait, then run the command again. Windows: download and install from git-scm.com, accepting the defaults, then close and reopen PowerShell.

Check: `git --version` prints a version number.

3 Install Python 3.10 or newer.

Both platforms: download the current release from python.org/downloads and install it. On Windows, tick "Add python.exe to PATH" on the first installer screen. Close and reopen the terminal afterwards.

Check: macOS `python3 --version`, Windows `py -3 --version`, prints 3.10 or higher.

4 Install Claude Code.

macOS: `curl -fsSL https://claude.ai/install.sh | bash`. Windows PowerShell: `irm https://claude.ai/install.ps1 | iex`. Close and reopen the terminal.

Check: `claude --version` prints a version.

5 Sign in to Claude Code.

Type `claude` and press Enter. A browser window opens; sign in with a Claude account that has a **Pro or Max** subscription (the free plan does not include Claude Code). If you would rather pay per use, skip the browser sign-in and instead set the API key from Section 7 as an environment variable; Claude Code will use it. Type `/exit` to leave.

Check: `claude doctor` reports no problems.

6 Get the project.

Choose a folder for your work, go there, and clone the repository:

```
cd ~/Documents                                (Windows: cd $HOME\Documents)
git clone https://github.com/robertdcurrier/korean_food_safety.git
cd korean_food_safety
```

Check: `ls` (Windows `dir`) lists `PLAN.md`, `CLAUDE.md`, `src`, `output`.

7 Create a private Python environment and install the project into it.

This keeps the project's libraries separate from everything else on your machine.

macOS	Windows
<pre>python3 -m venv .venv source .venv/bin/activate pip install -e .</pre>	<pre>py -3 -m venv .venv .venv\Scripts\activate pip install -e .</pre>

Your prompt now starts with `(.venv)`. You must run the *activate* line again every time you open a new terminal for this project.

Check: `python -m unittest discover -s tests` ends with `OK`.

8 Run the whole pipeline with no keys.

```
python -m kfs all
```

You will see it fetch five sample rows per service, say that it is skipping enrichment because there is no Anthropic key, geocode six districts, and build the page.

Check: the last line says `open ../output/index.html`.

9 Open the result.

macOS: `open output/index.html`. Windows: `start output\index.html`. Or find the file in Finder or Explorer and double-click it.

Check: a page titled K-식품안전 위치 with ten cases, a map of Korea with markers, and a working English button. The severity badges all say 미평가 / Not rated: that is correct, because no AI has run yet.

IF THE MAP AREA IS BLANK OR GREY WITH NO OUTLINES

Your browser may be blocking local files from loading scripts. Try a different browser (Chrome, Edge, Firefox and Safari all work), and make sure you are connected to the internet so the map library can load. Province outlines are inside the file and need no connection; street tiles do.

7. Getting your two keys

A key is a long secret string that tells a service who is asking. This project uses two, and both go into one small text file called `.env` in the project folder. Git is configured never to upload that file.

THREE RULES FOR EVERY KEY

1. Copy it once, at creation, into `.env`. Most sites show it only once.
2. Never paste it into chat, email, a slide, a screenshot, or any file that is committed to Git.
3. If you think it has leaked, delete it on the site and make a new one. Keys are free to replace.

7a. The Anthropic API key (unlocks the AI enrichment)

1 Create a Console account.

Go to **platform.claude.com** and sign up or sign in. This is the developer console, separate from the claude.ai chat app, and a Pro subscription does *not* include API credit.

2 Add credit.

Open **Settings** → **Billing** and add a payment method or buy a small amount of prepaid credit. US\$5 is plenty for everything in this guide; the full dataset costs about US\$9 to enrich once.

3 Create the key.

Open **Settings** → **API keys** → **Create key**. Give it a name such as *kfs-laptop* and, if offered, an expiry date. The key starts with `sk-ant-` and is shown once. Copy it now.

4 Put it in `.env`.

In the project folder, copy the example file and edit it:

```
cp .env.example .env          (Windows: copy .env.example .env)
```

Open `.env` in any plain-text editor (TextEdit in plain-text mode, Notepad, or VS Code) and make the first line exactly, with no spaces and no quotes:

```
ANTHROPIC_API_KEY=sk-ant-your-key-here
```

The name on the left matters. On the day this was built, the key was pasted in alone, without the `ANTHROPIC_API_KEY=` part, and the program could not see it. The program now warns about that mistake, but it is the single most common one.

5 Run the enrichment and rebuild.

```
python -m kfs enrich
python -m kfs build
```

Check: the enrich step reports 10/10 with a token count. Reload the page: badges now read 높음 / 보통, cases carry an "AI 요약", the hazard chart has several bars, and the map markers are coloured.

7b. The MFDS 식품안전나라 key (unlocks the full dataset)

Without this key you have five records per service. With it, several hundred, and the map and chart become genuinely informative. Registration is designed for Korean residents and uses the usual identity verification, so for you this is the easy key.

1 Go to the portal.

www.foodsafetykorea.go.kr/api (식품안전나라 → 공공데이터활용).

2 Sign up and log in.

회원가입, then 로그인. The site states that both are required before any Open-API use.

3 Find the services.

Click 데이터공개, then any of 분류별, 제공기관별 or 유형별. Tick the box for 회수·판매중지 정보 (I0490) and for 검사부적합(국내) (I2620). You can tick several at once.

4 Apply.

Click the **Open-API 이용신청** button at the top right, agree to the 공공데이터활용 이용약관, and for each service choose a usage type (application development is accurate), individual or corporate, and write a sentence of purpose in 활용목적. The field asks for detail; one honest sentence about a bilingual food-safety research tool is enough. Click 신청.

5 Collect the key.

Approval is automatic. Open 인증키 관리 → 인증키 신청 현황. You get one key for your account; it is issued once, and you add more services to it later by repeating step 3 and 4.

6 Add it to .env

as a second line:

```
MFDS_API_KEY=your-key-here
```

7

Fetch everything, enrich it, rebuild.

Enrichment of the full set takes a few minutes and about US\$9. If you want to try a smaller slice first, cap it:

```
python -m kfs fetch
python -m kfs enrich --max 50
python -m kfs geocode
python -m kfs build
```

Run `python -m kfs enrich` again later without `--max` to finish the rest; it only sends records it has not seen.

Check: the first tile on the page shows hundreds of cases, the province filter lists most of Korea, and the footer no longer mentions the sample key.

Daily limits per service apply (I0490: 2000 calls, I2620: 500). One full fetch uses one call per 1000 rows, so this is not a constraint in practice. Each service has its own permitted filter fields; see `docs/MFDS_API.md`.

8. Working with Claude Code on this project

This is where the method becomes a habit. Open a terminal in the project folder, activate the environment, and type `c laude`. You are now in a conversation with the builder, and it has already read `CLAUDE.md`.

Your first three prompts

1. *Read PLAN.md, CLAUDE.md and docs/METHODOLOGY.md, then explain to me in plain language what this project does and how the pieces fit. Do not change anything.*
2. *Show me the system prompt that the enrichment step sends to Claude, and the ten hazard categories. I am a food safety scientist; I want to review them.*
3. *Open the page in my browser.*

Nothing was built yet, and you already know more about the repository than most people who clone a project ever do.

A template for asking for changes

CONTEXT · GOAL · CONSTRAINTS · ASK

Context. Who you are and what the repo is. One or two sentences, or "you have already read PLAN.md and CLAUDE.md".

Goal. One sentence. What will be true when this is done.

Constraints. What must not break: the no-keys path, the bilingual labels, the tests, the 79-column rule.

Ask. "Report back with a plan before changing code." Then, after you agree, "go ahead".

Example, for the first exercise in Section 9:

I am a food safety scientist learning to work with you on this repository; you have read PLAN.md and CLAUDE.md. I want to add the MFDS administrative-dispositions service I2630 (행정처분) as a third source alongside recalls and inspections, so that enforcement actions appear on the same map and list. Keep the no-keys path working, keep every label bilingual, keep the tests passing, and update PLAN.md and docs/MFDS_API.md to match. Before you change any code, fetch a few real I2630 rows with the sample key, show me the fields, and give me a short plan.

Reviewing what it did

- **Read the summary it gives you.** It should say what changed and what it verified. If it says tests pass, you may ask it to show you the output.
- **Look at the diff.** Ask: *show me a summary of every file you changed and why.* You do not need to read code to notice when a change touched something unrelated.
- **Run it yourself.** `python -m kfs build` and open the page. Your eyes are the final test, and in this project they are the expert's eyes.

- **Commit when happy.** Ask: *commit this with a clear message*. Git keeps every version, so nothing is ever lost.

When it goes wrong

Paste the error message, all of it, and say what you were doing. "I ran python -m kfs enrich and got this:" followed by the text is a complete bug report. Do not paraphrase errors. Do not retype them. Copy and paste.

A HABIT THAT PAYS FOR ITSELF

End every working session with: *Update PLAN.md with what we decided today, add today's prompts to docs/PROMPT_LOG.md, and commit*. The repository then explains itself to your future self, your partner, and the next version of Claude.

9. Exercises, in order of difficulty

Each one is a complete conversation with Claude Code using the template above. The first two are mostly the builder's work; the last two are mostly yours.

- 1 Add a third source.**

Service I2630, 행정처분, administrative dispositions. It has the same address field, so the map and geocoder already work. Mostly plumbing; a good first exercise because the shape of the change is already visible in the code for the other two services.
- 2 Add a date-range filter to the page.**

The MFDS API accepts a `CRET_DTM` filter and the records already carry a clean date. Teaches how a change flows from API to template.
- 3 Street-level map points.**

Replace the free OpenStreetMap geocoder with the Kakao Local API, which needs a free Korean developer key and returns exact coordinates. Teaches swapping one external dependency for another behind the same interface.
- 4 A weekly bilingual digest.**

Ask Claude, from the enriched records, to write one paragraph at the top of the page: what happened this week, what mattered, in both languages. Teaches a second, different use of the API in the same app, and raises the question of what "mattered" means, which is yours to answer.
- 5 An evaluation.**

Take twenty records. Without looking at the AI's answers, write your own hazard category and severity for each. Then compare. Where you disagree, decide whether the rubric in the system prompt is wrong, the model is wrong, or you were. Tighten the prompt and re-run those twenty (delete their entries from `data/enriched.json` first). **This is the exercise that turns a demo into a tool, and no one but a domain expert can do it.**

10. Beyond food safety: the same recipe in any field

Nothing in the method is about food. Strip the nouns out of this repository and what remains is a recipe that any subject-matter expert can follow with any public data source. It has five ingredients:

1. **A public API the expert already knows about.** Every field has one. The expert's advantage is knowing which of the hundreds of datasets actually matters, and what the field names mean.
2. **One judgement task that only the expert can specify and check.** Translate, classify, grade, flag, summarise. This is the only place the Claude API runs, and it is where the expert's rubric lives.
3. **Deterministic code, with tests, for everything else.** Parsing, joining, geocoding, drawing. Free, fast, verifiable.
4. **A single local HTML result.** Nothing to deploy, nothing to pay for, easy to send to a colleague.
5. **A repository that explains itself.** Plan, standards, method, prompt log, committed results. The repo is the teaching artefact; the app is the evidence.

The same shape in other fields

Expert	Public API (all free, most need no key)	The one judgement step for Claude	Local HTML result
Botanist, ecologist	GBIF occurrence API; iNaturalist observations	Flag records whose location, date or habitat are implausible for the species; summarise what is unusual, in two languages	Map of sightings with an "needs checking" layer
Marine biologist	OBIS occurrences; NOAA NDBC buoy data	Classify strandings or sightings by likely cause from free-text notes; grade confidence	Coastal map with a timeline
Epidemiologist, public-health officer	KDCA or WHO outbreak notices; national open-data portals	Extract pathogen, setting, exposure route and severity from narrative reports into fixed fields	Searchable outbreak table and map
Chemist, toxicologist	PubChem; EPA CompTox; ECHA	Translate regulatory text into a hazard class and a plain-language one-liner per substance	Filterable substance list
Agronomist	KMA or NOAA weather; FAO crop data; national pest alerts	Turn pest and weather bulletins into a risk level per crop and region, with a reason	Regional risk map by week
Veterinarian	WOAH animal-disease events; national livestock portals	Classify disease events by transmission risk and trade impact; bilingual summary	Event map and digest

In every row the expert supplies the categories, the grading rubric and the twenty-record evaluation from Exercise 5. The partner supplies the terminal. Claude Code supplies the rest.

What you would actually change in this repository

File	Change
<code>src/kfs/mfds.py</code>	Replace with a client for your API: the URL shape, paging, and a <code>normalize()</code> that maps its field names onto a clean record. Probe it with <code>curl</code> first, exactly as in Section 5.
<code>src/kfs/enrich.py</code>	Rewrite the <code>SYSTEM</code> prompt and the output schema for your judgement task. Your categories, your rubric, your two languages or one.
<code>templates/report.html</code>	Change the label table at the top and whatever the map and chart should show. Keep the structure.
<code>src/kfs/geocode.py</code>	Keep if your records have addresses; drop if they already have coordinates, as most biodiversity APIs do.
<code>PLAN.md</code> , <code>CLAUDE.md</code> , <code>docs/</code>	Rewrite the plan first, keep the standards, start a fresh prompt log.
Everything else	Unchanged: the CLI, the caching, the no-keys path, the tests pattern, the build step, this guide's structure.

THE PROMPT THAT STARTS THE NEXT PROJECT

Open Claude Code in a clone of this repository and say: *"I am a [your field] scientist. I want to build the same kind of tool as this repository, but for [your data source], where the judgement task is [one sentence]. Read PLAN.md, CLAUDE.md and docs/METHODOLOGY.md, then probe the new API with curl, show me real records, and report back with a plan before changing anything."* That is the whole briefing. It took Section 5 of this guide to explain why it works; it takes thirty seconds to type.

11. When something goes wrong

Symptom	Likely cause and fix
command not found: python or git or claude	Not installed, or the terminal was open before you installed it. Close and reopen the terminal. On Windows make sure Python was installed with "Add to PATH".
No module named kfs	The environment is not active. Run the <i>activate</i> line from Section 6 step 7, in the project folder, then try again.
enrich says <i>no Anthropic credential found</i> although you set one	Check <code>.env</code> is in the project folder (not in <code>src</code>), is named exactly <code>.env</code> , and the line reads <code>ANTHROPIC_API_KEY=sk-ant-...</code> with no spaces, quotes or leading <code>export</code> .
enrich fails with an authentication or credit error	The key was mistyped, has expired, or the Console has no credit. Make a new key and check Settings → Billing.
fetch returns only five rows per service with a key set	The MFDS key was not read or the service was not applied for. Confirm the <code>.env</code> line, and on the portal confirm both I0490 and I2620 appear under 인증키 신청 현황.
fetch reports <code>INFO-100</code> or <code>INFO-300</code>	Key not registered or invalid. Copy it again from 인증키 신청 현황.
Map shows outlines but no street detail	The tile server was unreachable or refused the request; the page dropped it on purpose and kept the outlines. The data is unaffected.
Map shows "Access blocked" tiles	An old copy of the page from before 9 Oct 2026. Run <code>git pull</code> and <code>python -m kfs build</code> .
Labels on the page look like odd English ("height", "commonly")	Your browser auto-translated the Korean. The page now asks browsers not to; if it persists, turn off translation for local files. Use the page's own English button.
Tests fail after a change Claude Code made	Paste the failing output back to Claude Code and ask it to fix the cause, not the test.
Anything else	Copy the whole error, open <code>claude</code> in the project, paste it, say what you ran.

12. Glossary

Terminal	A window where you type commands instead of clicking. Terminal on macOS, PowerShell on Windows.
Repository (repo)	A project folder whose entire history is tracked by Git. <code>git clone</code> copies one to your machine.
Commit / push	Save a named snapshot of the repo / upload snapshots to GitHub.
Python	The programming language the pipeline is written in.
Virtual environment (.venv)	A private folder of Python libraries for one project, so projects do not interfere.
API	A way for programs to ask a service for data over the internet, using a URL instead of a web page. The MFDS API answers with JSON; the Claude API answers with text or JSON.
Endpoint	One specific URL an API answers at.
Key (API key, 인증키)	A secret string that identifies you to an API. Lives in <code>.env</code> , nowhere else.
JSON	A plain-text format for structured data that both humans and programs can read. Every file in <code>data/</code> is JSON.
Token	The unit AI usage is billed in; roughly three quarters of a word in English, less in Korean. Ten records cost about seven thousand.
Prompt / system prompt	The text you send to a model. The system prompt is the standing instruction that frames every request.
Structured output	Asking the model to answer in an exact JSON shape rather than prose, so the program never has to interpret free text.
Model	A specific version of Claude. This project uses Claude Opus 5.5 (<code>claude-opus-5-5</code>) by default; change it with <code>KFS_MODEL</code> in <code>.env</code> .
Cache	Saved answers so the same question is never paid for twice. <code>data/enriched.json</code> and <code>data/geocache.json</code> .
Geocoding	Turning an address into latitude and longitude.
Claude Code	The builder: Claude running as a tool on your computer that reads, writes, and runs things on your instruction.
Claude API	The worker: Claude on Anthropic's servers, called by your program with a key.

Sources and attributions. Data: 식품의약품안전처 식품안전나라 공공데이터. Province boundaries: KOSTAT via the southkorea-maps project. Map tiles: Esri World Street Map. Map library: Leaflet. Model: Claude Opus 5.5 by Anthropic. This guide was drafted by Claude Code from the session that built the repository and reviewed by its author.